נ.נ. בתכנון משולב חומרה/תוכנה
אביב – 00460882
תרגול 2 – מתלכלכים עם Perf

אוהד איתן

2D array row major vs column major

2D array row major vs column major

Metric	Column major	Row major
Cycles	51.6 billion	20.5 b
Instructions	25.3 b	25.3 b
Cache References	3.08 b	120 m
Cache Misses	214.6 million (6.96%)	76.8 m (63.93%)
Time+-	10.8 [sec]	3.81 [sec]

What do we observe?

- Same output :)
- Both versions execute roughly the same number of instructions, meaning the **logical work** done is identical.
- The optimized code runs in less than half the cycles of the non-optimized one.
- This indicates better CPU efficiency, mainly because memory accesses are faster and more predictable.

- Let’s conclude that: the difference in performance is entirely due to **how memory is accessed**.

- Using the memcpy version results in approximately **25× fewer cache accesses**.
- This shows that the CPU can make better use of each cache line: by accessing contiguous memory, one cache line brings many needed elements at once.

But the miss rate is bad… Is it?

- The optimized version has a **higher cache miss percentage** 63% vs 7%. Why?

But the miss rate is bad. **Is it?**

- The optimized version has a **higher cache miss percentage** 63% vs 7%. Why?
- but **a much lower total number** of cache references and misses overall. Due to spatial locality.
- This might happen because the working set is large enough that cache lines get evicted quickly, but it doesn’t hurt overall performance since each memory fetch brings in more useful data at once.
- In row-major traversal, memory is accessed sequentially. A single cache line (typically 64 bytes) holds several adjacent integers (e.g., 16 ints at once), so each memory access loads useful data for several upcoming reads.

2D array better analysis on caches

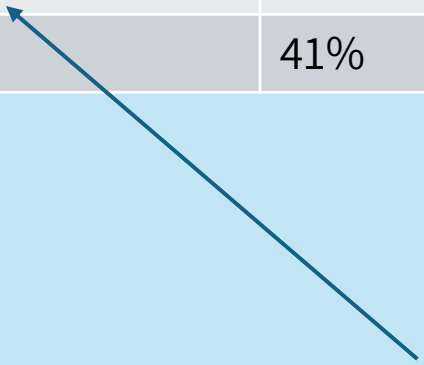
Metric	Column based	Row based
L1-dcache-loads	13.5 billion	13.5 b
L1-dcache-load-misses	1.8 b (13.8%)	124 m (0.92%)
Cache References	3.08 b	120 m
Cache Misses	214.6 million (6.96%)	76.8 m (63.93%)

Copy Memory

- Manual v.s. memcpy. Thoughts?
- If someone wrote it, maybe he knows better then you.
- Leverages hardware more efficiently

Copy Memory

metric	Manual Copy	memcpy
Time Elapsed	7.24 [sec]	5.96 [sec]
CPU Cycles	32.76 billion	26.98 b
Instructions Retired	47.5 b	32.0 b
Instructions per Cycle (IPC)	1.45	1.19
Branches	7.49 b	5.52 b
Backend Bound (stall time)	40.8%	45.3%
Bad speculation	41%	36%



Backend bound means that computation or memory access is the bottle neck. (was taken from man page)

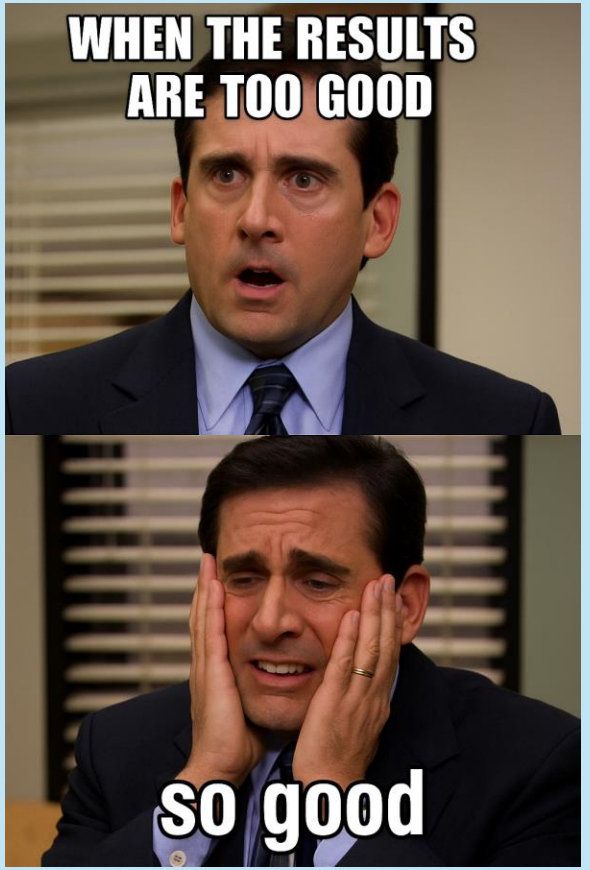
What do we observe?

- The optimized version (memcpy) completed **~18% faster** than the manual copy (5.96s vs 7.24s).
- memcpy needed **15.5 billion fewer instructions** than the manual copy.
- This demonstrates how memcpy internally takes advantage of wide memory transfers. When buffers are aligned to cache lines (typically 64 bytes), using the SIMD unit is optimal, as it can move 64 bytes at once efficiently.
- Optimized code used **5.8 billion fewer CPU cycles**, making it more energy and performance efficient.

- Both versions show high "backend bound" percentages (>40%), meaning they spent a lot of time waiting for memory accesses.
  Even though `memcpy` is faster, both are ultimately **memory bandwidth limited**.
- The optimized code has slightly lower bad speculation (36% vs 41%), showing better branch prediction efficiency when not manually looping.

New Thread per Task vs Thread Pool

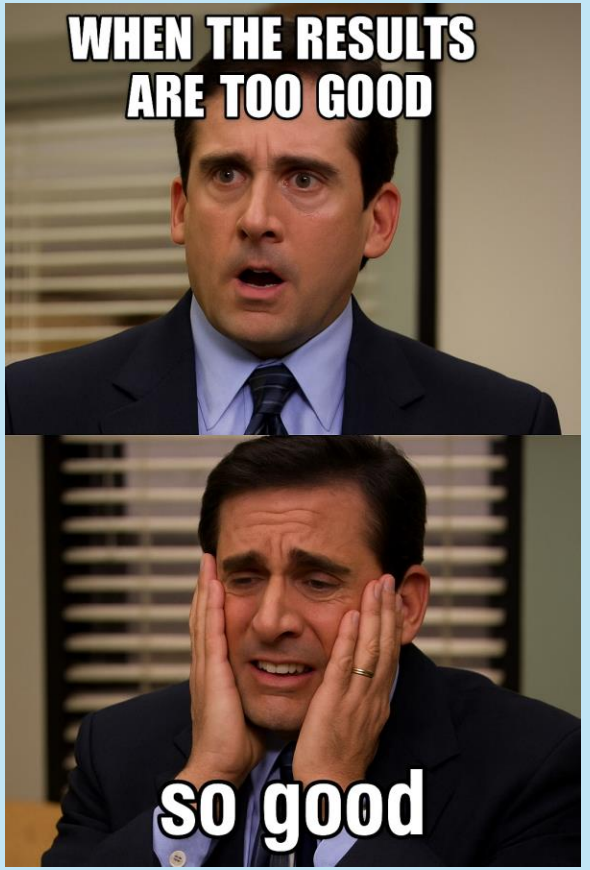
Disclaimer – this example is a bit too good to be true. I hope there is no mistake :)



New Thread per Task vs Thread Pool

Metric	New Thread per Task	Thread Pool
Task Clock (CPU time)	2,561 [ms]	41 ms
Context Switches	3,881	1,087
CPU Cycles	10.2 billion	35 million
Instructions Retired	10.1 billion	16 million
Elapsed Time	0.19 [sec]	1.27 [sec]
Backend Bound	34.5%	12.9%

Disclaimer – this example is a bit too good to be true. I hope there is no mistake :)



What do we see

- The **unoptimized** version finished faster **(lower wall time)** because you exploded the system with 1000 threads.
- **Is my example still useless?**

What do we see

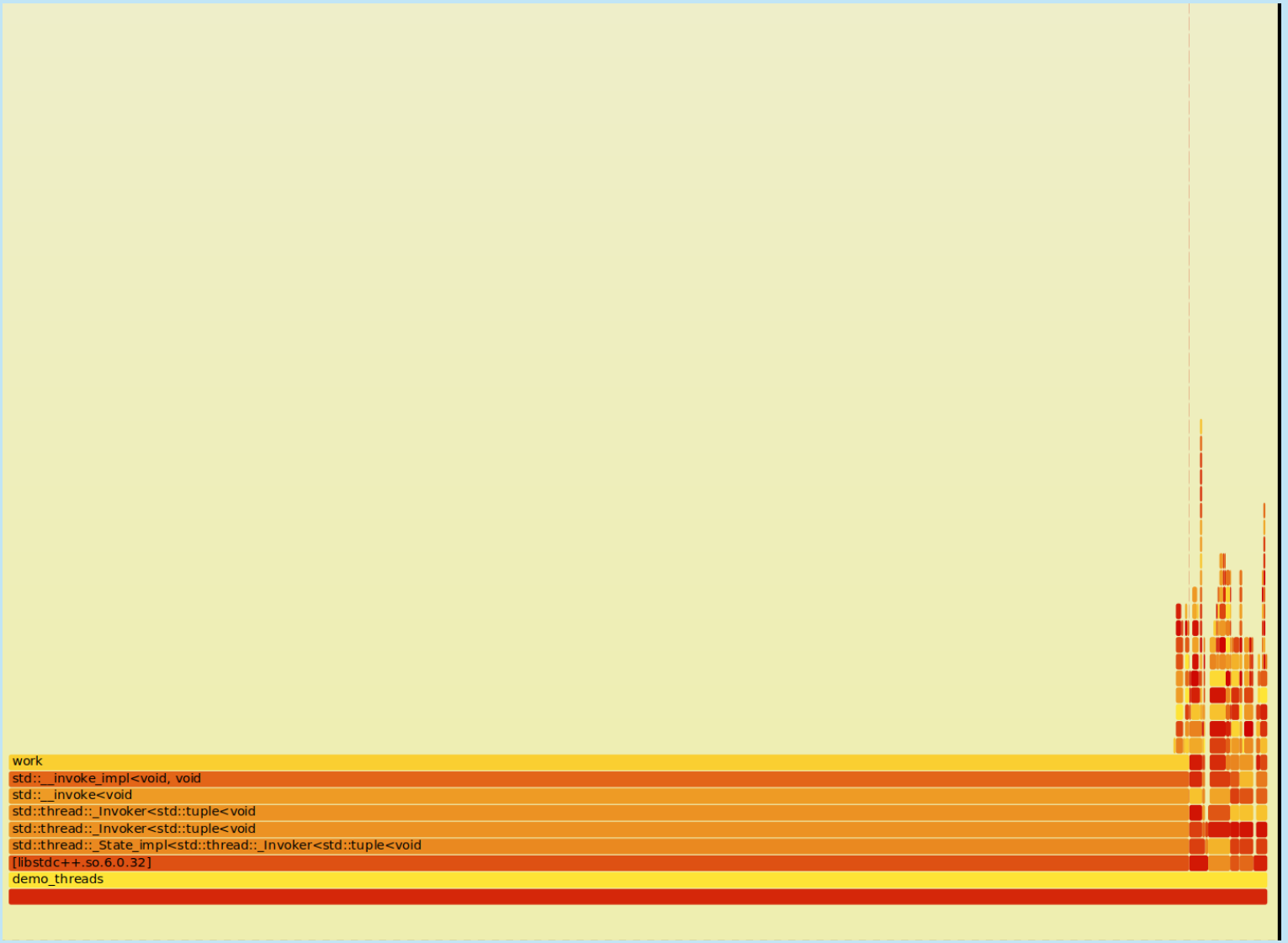
- The time difference is huge.
- Creating a new thread for every task causes **many context switches.** The CPU must keep saving/restoring thread states.
- Thread pool **reduced context switches by ~3.5×**.
- The output shows **massive overhead** was eliminated by using a thread pool.
- In the **bad version**, backend (memory, resources) stalls were high

- The **naive** version finished faster **(lower wall time)** because you exploded the system with 1000 threads.
- The Linux kernel scheduled them aggressively.
- You wasted **tons of CPU cycles** (thread creation, scheduling, context switching).
- CPU was overloaded but finished "faster" by brute force.

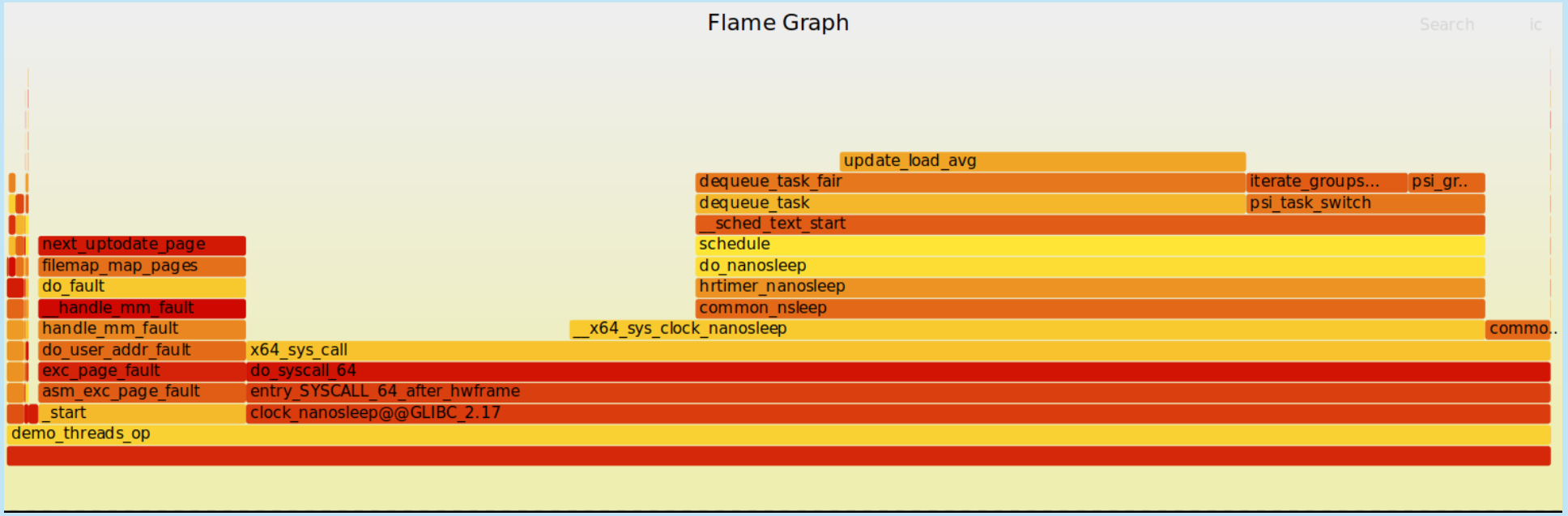
- The **optimized** (thread pool) version had **longer wall time**, because only 32 threads worked at the same time.
- Tasks were picked and executed **calmly**.
- CPU cycles were used **efficiently** for real work.
- No energy wasted on thread management.

Flame Graph

Not optimized



Optimized


Flame Graph
Search
ic
update_load_avg
dequeue_task_fair
iterate_groups...
psi_gr..
dequeue_task
psi_task_switch
__sched_text_start
next_uptodate_page
schedule
filemap_map_pages
do_nanosleep
do_fault
hrtimer_nanosleep
__handle_mm_fault
common_nsleep
handle_mm_fault
__x64_sys_clock_nanosleep
commo..
do_user_addr_fault
x64_sys_call
exc_page_fault
do_syscall_64
asm_exc_page_fault
entry_SYSCALL_64_after_hwframe
_start
clock_nanosleep@@GLIBC_2.17
demo_threads_op